

# Hackathon Problem Statements

## Industry-Oriented Internal Laboratory Assessment

**Participants:** MCA 2nd Year Students

**Hackathon Duration:** 5 Hours

**Development:** From Scratch

**AI Tools:** Permitted

## General Instructions

Each participant/team will be assigned **one problem statement**.

The objective is to build a functional **Minimum Viable Product (MVP)** within the allotted five-hour development period.

Students are expected to make their own decisions regarding:

- Technology stack
- Application architecture
- Database structure
- API design
- UI design
- Validation approach
- AI tools and techniques

The problem statements define the **business requirements**, but do not prescribe the implementation approach.

## Common Expectations

Every solution should demonstrate, wherever applicable:

- Functional user interface
- Backend/API
- Database
- Business logic
- Input validation
- Error handling
- Appropriate data relationships
- Basic testing
- Clean and understandable code

Students may use AI tools and open-source libraries.

# Table of Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Hackathon Problem Statements .....                                 | 1  |
| Problem Statement 1: Event Discovery & Registration Platform ..... | 3  |
| Problem Statement 2: Customer Support Ticketing System .....       | 6  |
| Problem Statement 3: AI Meeting Action Tracker.....                | 8  |
| Problem Statement 4: Smart Invoice Manager .....                   | 10 |
| Problem Statement 5: Candidate Matching Platform .....             | 12 |
| Problem Statement 6: Product Recommendation System.....            | 15 |
| Problem Statement 7: Business Expense Analyzer.....                | 17 |
| Problem Statement 8: API Health Monitoring Dashboard .....         | 19 |
| Problem Statement 9: Content Intelligence Platform .....           | 21 |
| Problem Statement 10: Service Booking Platform .....               | 23 |
| Common MVP Boundary .....                                          | 25 |

# Problem Statement 1: Event Discovery & Registration Platform

## 1. Business Scenario

Organizations conduct conferences, workshops, webinars, networking sessions and other events. Users need a simple way to discover relevant events and register for them, while organizers need to manage events and participant registrations.

## 2. Objective

Build an event management platform with two types of users:

- **Event Organizer**
- **Event Participant**

The platform should support the complete basic journey from event creation to participant registration.

## 3. Functional Requirements

### Participant

Participants should be able to:

- View available events.
- Search events.
- Filter events by category/date.
- View event details.
- Register for an event.
- Cancel their registration.
- View their registered events.

### Organizer

Organizers should be able to:

- Create an event.
- Enter event name, description, date, time and location.
- Select an event category.
- Define maximum capacity.
- Edit event details.
- Cancel an event.
- View registered participants.
- View basic registration statistics.

## 4. Event Information

An event may contain:

- Event name
- Description
- Category
- Date
- Start/end time
- Location or online meeting information
- Organizer
- Registration capacity
- Registration status

## 5. Business Rules

- A participant cannot register for the same event more than once.
- Registration cannot exceed the defined capacity.
- A cancelled registration should release the seat.
- Cancelled events should not accept new registrations.
- Events should be displayed based on their status/date.

## 6. Expected MVP Screens

The application should provide appropriate screens for:

- Event listing
- Event details
- Registration
- My registrations
- Organizer event management
- Participant/registration dashboard

Students may combine or structure these screens differently.

## 7. Minimum Data Entities

The solution is expected to manage information corresponding to:

- User
- Event
- Registration

Additional entities may be created where required.

## 8. Out of Scope

The following are not required:

- Real payment gateway
- Email/SMS integration
- Complex ticket generation
- Real-time notifications
- Multi-organization enterprise management

# Problem Statement 2: Customer Support Ticketing System

## 1. Business Scenario

Companies receive customer complaints, questions and service requests. Support teams need a centralized system to track these requests from creation through resolution.

## 2. Objective

Build a support ticket management platform with:

- Customers
- Support agents
- Ticket management

## 3. Functional Requirements

### Customer

Customers should be able to:

- Create a support ticket.
- Enter subject and description.
- Select an issue category.
- View their tickets.
- View ticket status.
- View responses/history.
- Add additional information to an open ticket.

### Support Agent

Agents should be able to:

- View incoming tickets.
- Search/filter tickets.
- Assign tickets to agents.
- Change priority.
- Change status.
- Respond to customers.
- Resolve/close tickets.

## 4. Ticket Information

A ticket may contain:

- Ticket ID
- Customer

- Subject
- Description
- Category
- Priority
- Assigned agent
- Status
- Creation date
- Last updated date
- Responses/history

## 5. Business Rules

Suggested statuses:

**Open → In Progress → Resolved → Closed**

- Every ticket must have a status.
- Every ticket should have a priority.
- Closed tickets should not be modified without reopening.
- High-priority tickets should be clearly identifiable.
- Ticket history should retain previous responses/status changes.

## 6. Expected MVP Screens

- Customer ticket list
- Create ticket
- Ticket details/conversation
- Agent ticket dashboard
- Ticket management
- Basic support dashboard

## 7. Minimum Data Entities

- Customer/User
- Agent
- Ticket
- Category
- Ticket Response/History

## 8. Out of Scope

- WhatsApp/email integration
- Voice support
- Complex SLA management
- Multi-company enterprise configuration
- Production-grade notification infrastructure

# Problem Statement 3: AI Meeting Action Tracker

## 1. Business Scenario

Business meetings generate decisions, tasks and follow-ups. Important actions can easily get lost in meeting notes.

A company wants a simple system to convert meeting information into structured actions and track their completion.

## 2. Objective

Build an application for managing meetings, participants and action items.

## 3. Functional Requirements

Users should be able to:

- Create a meeting.
- Enter meeting title, date and description.
- Add participants.
- Enter or paste meeting notes.
- Create action items.
- Assign an action to a participant.
- Set due dates.
- Set action status.
- View pending and completed actions.
- Filter actions by owner/status/due date.

## 4. Action Item Information

An action item may contain:

- Description
- Owner
- Due date
- Priority
- Status
- Source meeting
- Creation date

## 5. AI Capability

The application may use AI to process meeting notes.

The AI functionality should be capable of identifying useful information such as:

- Summary

- Decisions
- Potential action items
- Suggested owners
- Suggested deadlines

The user should remain able to review and modify AI-generated information.

## 6. Business Rules

- Every action item must have an owner.
- Every action item must have a status.
- Overdue actions should be identifiable.
- Completed actions should remain available in history.
- AI-generated information should not automatically overwrite user-entered information without review.

## 7. Expected MVP Screens

- Meeting list
- Meeting details/notes
- Action item list
- Action item details/edit
- AI processing interface
- Pending/overdue action dashboard

## 8. Minimum Data Entities

- User
- Meeting
- Participant
- Action Item

## 9. Out of Scope

- Live meeting transcription
- Video conferencing
- Calendar integration
- Real-time collaboration
- Enterprise authentication

# Problem Statement 4: Smart Invoice Manager

## 1. Business Scenario

Businesses receive invoices from multiple vendors. Maintaining invoice information manually can make it difficult to track payments, identify overdue invoices and understand spending.

## 2. Objective

Build an invoice management application for maintaining and tracking business invoices.

## 3. Functional Requirements

Users should be able to:

- Add vendors.
- Create invoices.
- Add invoice line items.
- Record invoice number.
- Record invoice date and due date.
- Record taxes and total amount.
- Track payment status.
- Search/filter invoices.
- View invoice details.
- View basic invoice statistics.

## 4. Invoice Information

An invoice may contain:

- Vendor
- Invoice number
- Invoice date
- Due date
- Line items
- Quantity
- Unit price
- Tax
- Total amount
- Payment status

Suggested payment statuses:

**Unpaid / Partially Paid / Paid**

## 5. Business Rules

- Invoice number should be unique for a vendor.

- Total amount should correspond to line items and applicable taxes.
- Overdue unpaid invoices should be identifiable.
- Paid invoices should be distinguishable from unpaid invoices.

## 6. AI Capability

Students may implement AI-based extraction from invoice content.

The system may accept invoice information as:

- Text
- Image/document

AI may be used to identify:

- Vendor
- Invoice number
- Dates
- Tax
- Total amount
- Line items

Extracted information should be presented for verification before being saved.

## 7. Expected MVP Screens

- Vendor list
- Invoice list
- Add/edit invoice
- Invoice details
- Invoice dashboard
- AI extraction/review screen, if implemented

## 8. Minimum Data Entities

- Vendor
- Invoice
- Invoice Item
- Payment Status

## 9. Out of Scope

- Actual payment processing
- Accounting software integration
- GST filing
- Bank integration
- Automated email processing

# Problem Statement 5: Candidate Matching Platform

## 1. Business Scenario

Recruiters often receive many applications for a job opening. Reviewing every candidate manually can be time-consuming.

A recruitment platform is required to maintain candidate information and help recruiters identify relevant candidates.

## 2. Objective

Build a simplified recruitment and candidate management platform.

## 3. Functional Requirements

### Recruiter

Recruiters should be able to:

- Create job openings.
- Define job description.
- Specify required skills.
- Define minimum experience.
- View candidate profiles.
- Search/filter candidates.
- View applications.
- Shortlist candidates.

### Candidate

Candidates should be able to:

- Create their profile.
- Add skills.
- Add education.
- Add experience.
- Provide resume information.
- Apply for available jobs.
- View their applications.

## 4. Candidate Information

A candidate profile may include:

- Name
- Education
- Skills

- Experience
- Resume information
- Contact information

## 5. Job Information

A job may contain:

- Job title
- Description
- Required skills
- Minimum experience
- Location
- Employment type
- Status

## 6. Business Rules

- A candidate cannot apply for the same job more than once.
- Recruiters should be able to filter candidates based on defined criteria.
- Closed/inactive jobs should not accept new applications.
- Candidate applications should maintain their status.

## 7. AI Capability

The platform may use AI to compare a candidate profile/resume with a job description.

The system may generate:

- Match percentage
- Matching skills
- Missing skills
- Short explanation of the match

The recruiter should be able to review the AI output before making a decision.

## 8. Expected MVP Screens

- Job listing
- Job creation/edit
- Candidate profile
- Candidate listing
- Application management
- Candidate-job matching screen

## 9. Minimum Data Entities

- Candidate
- Job

- Skill
- Application
- Recruiter/User

## 10. Out of Scope

- Video interviews
- Payroll
- Background verification
- Job portal integrations
- Production-grade resume parsing infrastructure

# Problem Statement 6: Product Recommendation System

## 1. Business Scenario

Online businesses have large product catalogues. Showing relevant products based on customer behaviour can improve product discovery.

## 2. Objective

Build a simplified product discovery and recommendation platform.

## 3. Functional Requirements

Users should be able to:

- Browse products.
- Search products.
- Filter by category.
- View product details.
- Mark products as favourites.
- View their recently viewed products.
- Receive product recommendations.

Administrators should be able to:

- Add products.
- Edit products.
- Categorize products.
- Manage product availability.

## 4. Product Information

Products may contain:

- Name
- Description
- Category
- Price
- Image
- Availability

## 5. User Activity

The application should record activities such as:

- Product view
- Favourite
- Category interaction

These activities should be used to generate relevant recommendations.

## 6. Business Rules

- Products should belong to categories.
- User activity should be associated with the appropriate user.
- Unavailable products should not be recommended.
- Recommendations should be based on available user activity or category preferences.

## 7. AI Capability

Students may use AI to improve recommendation logic or generate explanations.

For example:

“Recommended because you recently viewed products in this category.”

The explanation should be based on the user’s actual recorded activity.

## 8. Expected MVP Screens

- Product catalogue
- Product details
- Favourites
- Recently viewed
- Recommendations
- Product administration

## 9. Minimum Data Entities

- User
- Product
- Category
- User Activity

## 10. Out of Scope

- Real payment gateway
- Order fulfilment
- Logistics
- Large-scale recommendation infrastructure
- Real-time personalization across millions of users

# Problem Statement 7: Business Expense Analyzer

## 1. Business Scenario

Businesses need visibility into spending patterns to understand where money is being spent and identify unusual transactions.

## 2. Objective

Build an expense management and analytics application for a small business.

## 3. Functional Requirements

Users should be able to:

- Add income transactions.
- Add expense transactions.
- Categorize transactions.
- Record vendor/payee.
- Enter transaction date and amount.
- Add notes.
- Edit/delete transactions.
- Search/filter transactions.

The system should provide:

- Monthly spending summary
- Category-wise analysis
- Vendor-wise analysis
- Income vs. expense summary
- Basic charts

## 4. Business Rules

- Every expense should have a category.
- Every transaction should have a date and amount.
- Reports should respect the selected date range.
- Users should be able to filter by category/vendor/date.
- Deleted transactions should not appear in active reports.

## 5. Analytics

The dashboard should provide meaningful information such as:

- Total income
- Total expenses
- Net balance
- Highest spending categories

- Highest spending vendors
- Monthly spending trend

## 6. AI/Intelligence Capability

Students may implement unusual-expense detection.

The system should identify transactions that differ significantly from historical spending.

For example:

“Office Supplies expense of ₹45,000 is significantly higher than the historical average.”

The system should display the transaction and the basis for the flag.

## 7. Expected MVP Screens

- Transaction list
- Add/edit transaction
- Categories
- Dashboard
- Expense analysis
- Unusual transaction view

## 8. Minimum Data Entities

- User
- Transaction
- Category
- Vendor

## 9. Out of Scope

- Bank integration
- Actual accounting
- Tax filing
- Payment processing
- Full enterprise accounting functionality

# Problem Statement 8: API Health Monitoring Dashboard

## 1. Business Scenario

Applications depend on multiple APIs and services. Development teams need to know whether these services are available and responding within acceptable limits.

## 2. Objective

Build a simple API monitoring application that tracks the health and response performance of registered API endpoints.

## 3. Functional Requirements

Users should be able to:

- Register an API endpoint.
- Define a response-time threshold.
- Trigger/check an API.
- View HTTP status.
- View response time.
- View current API status.
- View previous health-check results.

The dashboard should provide an overview of all monitored APIs.

## 4. API Information

An endpoint may contain:

- API name
- URL
- HTTP method
- Expected response threshold
- Active/inactive status

## 5. Health Check Information

Each check may record:

- Timestamp
- HTTP status
- Response time
- Success/failure
- Error information

## 6. Business Rules

- Every health check should produce a recorded result.

- Failed requests should be identified.
- Response times exceeding the configured threshold should be flagged.
- Inactive APIs should not be checked.
- Historical results should remain available.

## 7. Expected MVP Screens

- API list
- Add/edit API
- API details
- Health-check history
- Overall monitoring dashboard

## 8. Minimum Data Entities

- API Endpoint
- Health Check
- Response History

## 9. Out of Scope

- Distributed monitoring
- Production alerting systems
- SMS/email notification infrastructure
- Kubernetes/container monitoring
- Advanced observability/tracing

# Problem Statement 9: Content Intelligence Platform

## 1. Business Scenario

Organizations produce articles, reports, announcements and other textual content. Teams often need to summarize and repurpose the same content for different audiences and channels.

## 2. Objective

Build an AI-assisted content intelligence application that processes textual content and produces useful structured outputs.

## 3. Functional Requirements

Users should be able to:

- Enter or paste content.
- Save content.
- View previously submitted content.
- Edit content.
- Generate a summary.
- Extract keywords.
- Identify topics/categories.
- Add or edit tags.

## 4. AI Capabilities

The system should use AI to process the content.

Possible outputs include:

- Short summary
- Key points
- Keywords
- Topics/categories
- Suggested tags

Generated information should be presented to the user for review.

## 5. Content Transformation

The application should be designed so that the same source content can be transformed into different formats.

Possible formats include:

- FAQ
- Short social-media post

- Executive summary

## 6. Business Rules

- Content should be associated with a user.
- Generated outputs should be editable.
- Previously processed content should remain accessible.
- The original content should remain separate from generated content.

## 7. Expected MVP Screens

- Content creation
- Content list/history
- Content details
- AI processing
- Generated output
- Edit/save output

## 8. Minimum Data Entities

- User
- Content
- Generated Output
- Tags/Keywords

## 9. Out of Scope

- Video processing
- Automated publishing to social media
- Content scheduling
- Enterprise content management
- Large-scale document storage

# Problem Statement 10: Service Booking Platform

## 1. Business Scenario

Professional service businesses need to manage appointments between customers and service providers.

Examples include consultants, repair services, salons, coaching services and other appointment-based businesses.

## 2. Objective

Build a service booking platform that allows customers to view available services and book appointments.

## 3. Functional Requirements

### Customer

Customers should be able to:

- View available services.
- View service details.
- View available time slots.
- Select a service.
- Book an appointment.
- Cancel an appointment.
- View booking history.

### Service Provider

Service providers should be able to:

- Create/manage services.
- Define available time slots.
- View bookings.
- Cancel/reschedule bookings.

## 4. Service Information

A service may contain:

- Service name
- Description
- Duration
- Price
- Service provider
- Availability

## 5. Booking Information

A booking may contain:

- Customer
- Service
- Provider
- Date
- Time slot
- Status
- Booking timestamp

## 6. Business Rules

- A time slot cannot be booked by multiple customers.
- Customers cannot book unavailable slots.
- Cancelled bookings should release the slot.
- A booking should have a clear status.
- Service providers should be able to view their upcoming bookings.

## 7. Expected MVP Screens

- Service listing
- Service details
- Available slots
- Booking screen
- My bookings
- Provider dashboard
- Booking management

## 8. Minimum Data Entities

- Customer
- Service Provider
- Service
- Time Slot
- Booking

## 9. Out of Scope

- Payment gateway
- Video consultation
- SMS/WhatsApp integration
- Complex recurring appointments
- Multi-location enterprise scheduling

# Common MVP Boundary

The objective is **not to build a production-ready application**.

Within the five-hour Hackathon, students should prioritize:

**Functional Core → Business Logic → Data Integrity → Basic UI → Testing**

Additional features may be implemented if time permits, but they should not compromise the core functionality.

Students are free to choose their technology stack, architecture, database model and implementation approach.

**End of Problem Statements**